Workshop II Practical — Shared State, Indexing & PTBs

Quick recap

- We published a Move package and received an `AdminCap` in `init` (publisher-owned).
- We minted a `Grant` object for a student and emitted a `GrantMinted` event.
- We verified the new owned object on-chain.

What's next

We need a way to look up a student's current grant. This is important for portals and verifiers to render status without replaying history.

- Missing piece: a shared, canonical index mapping `student -> grant_id` and a way to update it whenever we mint.
- Why it matters:
  - Portals and verifiers need O(1)-style lookups to render "current grant" without replaying history.
  - Multiple apps can coordinate on one source of truth instead of bespoke caches.
  - A PTB lets us mint and index atomically — either both succeed or neither does — reducing partial failures and data drift.

What you'll build next:

- Share an on-chain `Registry` object that anyone can read
- Add a write-gated `index_grant` entry for admins to insert student → grant
- Emit `GrantIndexed` when the index changes
- Chain actions (mint → index) in a single PTB

What we're building (mental model first)

- A `Registry` shared object that holds a table from `address -> ID`. Shared means many users can touch it.
- A `GrantIndexed` event for analytics and UI updates. Will be emitted on every index change.
- An `index_grant` function that "inserts" a student's latest grant ID.
- A `mint_return_id` entry so we can capture a newly created `Grant` ID and pass it to `index_grant` in the same PTB.

WHAT IS A SHARED OBJECT?

Objects with shared ownership can be accessed and updated by many parties over time. They're ideal for registries, markets, and leaderboards. Learn more: Shared vs Owned

WHAT IS A TABLE?

Tables are scalable, typed key–value stores built on dynamic fields. They're a good fit when you don't know keys ahead of time. Learn more: Dynamic Fields: Tables & Bags

WHAT IS A PTB?

A Programmable Transaction Block composes multiple calls and object ops into one atomic transaction. Great for "do X then Y if X succeeded" patterns. Learn more: Use Programmable Transaction Blocks and the CLI PTB Reference

Carry-overs from Workshop I

Export these for convenience (reuse from last session):

```
export PKG_ID=<0x...from Workshop I>
export ADMIN_CAP_ID=<0x...admin cap object id>
export PUBLISHER_ADDR=<0x...publisher address>
export STUDENT_ADDR=<0x...student address>
export AMOUNT=1000
```

Model the Registry (start small)

Create `sources/registry.move`. Begin with the module, imports, and data types.

```
module scholarflow::registry {
    use iota::object::{Self, UID, ID};
    use iota::tx_context::{Self, TxContext};
    use iota::transfer;
    use iota::event;
    use iota::table::{Self as table, Table};
    use std::option;

    /// The shared Registry with an attached index of student -> grant
ID.
    public struct Registry has key {
        id: UID,
        by_student: Table<address, ID>,
    }

    /// Emitted when a student is indexed with a grant ID.
    public struct GrantIndexed has copy, drop, store {
        student: address,
        grant_id: ID,
    }
}
```

> **BIG PICTURE**
>
> This defines a single, shared "roster" where each student address points to their latest `grant_id`. UIs and indexers can query once and render state without chasing historical logs.

Make it shared

Append a `create` entry to instantiate and share the Registry.

```
    /// Create and share a Registry.
    public entry fun create(
        _cap: &scholarflow::grant::AdminCap,
        ctx: &mut TxContext
    ) {
        let reg = Registry {
            id: object::new(ctx),
            by_student: table::new<address, ID>(ctx),
        };
        transfer::share_object(reg);
    }
```

WHAT IS WRITE-GATING?

We'll require `&AdminCap` for mutating entries. Think of it as an admin badge controlling who can write to shared state.

BIG PICTURE

Sharing the registry allows many transactions to see and update the same object over time, which is perfect for global indexes, markets, and registries.

Your turn: index a student's grant

The `index_grant` entry should insert `student -> grant_id` and emit a `GrantIndexed` event. Add the signature and start with a placeholder; then implement.

```
/// insert mapping student -> grant_id and emit an event.
public entry fun index_grant(
    reg: &mut Registry,
    student: address,
    grant_id: ID,
    _cap: &scholarflow::grant::AdminCap
) {
    // TODO: table::contains / table::remove / table::insert
    // Emit event::emit(GrantIndexed { student, grant_id });
    abort 0;
}
```

Tip: Remove an existing mapping first (if present), then insert the latest `grant_id`.

BIG PICTURE

inserting keeps the registry idempotent and always current for a given student. Frontends only need one read to show the latest grant.

Chain actions: return the minted ID

To compose with a PTB, we'll expose a mint that returns the newly created `Grant` ID. Add this entry to your existing `scholarflow::grant` module.

UPGRADE COMPATIBILITY

- Do NOT change the existing `mint` function's signature from Workshop I. Upgrades must keep all existing public/entry function signatures identical.
- Instead, add a new entry function `mint_return_id` (below) alongside your existing `mint`.
- Also avoid changing struct layouts or abilities; adding new functions or modules is fine.

```
module scholarflow::grant {
    /// ... existing imports, structs, events, and functions ...

    /// Mint a grant and return its ID so callers (e.g., a PTB) can
chain actions.
    public entry fun mint_return_id(
        student: address,
        amount: u64,
        _cap: &AdminCap,
        ctx: &mut TxContext
    ): ID {
        let grant = Grant { id: object::new(ctx), student, amount };
        let gid: ID = object::id(&grant);
        event::emit(GrantMinted { student, amount, grant_id: gid });
        transfer::transfer(grant, student);
        gid
    }
}
```

BIG PICTURE

Returning the freshly minted `Grant` ID lets downstream steps (like indexing) reference it immediately, avoiding extra RPCs or guesswork.

Build and upgrade

AUTOMATED ADDRESS MANAGEMENT

IOTA tracks package addresses per environment in `Move.lock`, so you don't need `published-at` in `Move.toml`. Keep your package's named address at `0x0` and rely on automated address management. If migrating an older package, see: Automated Address Management.

```
iota move build
```

```
iota client upgrade --package $PKG_ID --upgrade-cap
<0x...UpgradeCapID> --gas-budget 150000000
```

WHAT IS AN UPGRADE?

Upgrades let you evolve a package without changing its on-chain address, while enforcing compatibility rules. Learn more: Package Upgrades

BIG PICTURE

Upgrading preserves your package address so clients don't need to be reconfigured and links don't break.

Create the Registry and capture its ID

```
iota client call \
  --package $PKG_ID \
  --module registry \
  --function create \
  --args $ADMIN_CAP_ID \
  --gas-budget 80000000

# After the call, set:
# export REGISTRY_ID=<0x...registry object id>
```

BIG PICTURE

The `REGISTRY_ID` is the anchor for all future index updates. Capture it once; reuse it across tools and scripts.

Compose an atomic PTB: mint + index

Environment:

```
export PKG_ID
export ADMIN_CAP_ID
export REGISTRY_ID
export STUDENT_ADDR
export AMOUNT
```

Move CLI PTB example:

```
iota client ptb \
  --move-call "$PKG_ID::grant::mint_return_id" "$STUDENT_ADDR"
"$AMOUNT" @"$ADMIN_CAP_ID" \
  --assign minted_id \
  --move-call "$PKG_ID::registry::index_grant" @"$REGISTRY_ID"
"$STUDENT_ADDR" minted_id @"$ADMIN_CAP_ID" \
  --gas-budget 100000000
```

Expected output (events excerpt):

```
╭─────────────────────────────────────────────────────────────────────────────╮
│ Transaction Block Events                                                    │
├─────────────────────────────────────────────────────────────────────────────┤
│  ╭──                                                                        │
│  │ EventID: <event_id>:0                                                    │
│  │ PackageID: 0x<package_id>                                                │
│  │ Transaction Module: grant                                                │
│  │ Sender: 0x<sender_address>                                               │
│  │ EventType: 0x<package_id>::grant::GrantMinted                            │
│  │ ParsedJSON:                                                              │
│  │   ╭────────────┬──────────────────────────────────────────────────────╮│
│  │   │ amount     │ <amount>                                             │
│  │   ├────────────┼──────────────────────────────────────────────────────╮│
│  │   │ grant_id   │ 0x<grant_object_id>                                  │
│  │   ├────────────┼──────────────────────────────────────────────────────╮│
│  │   │ student    │ 0x<student_address>                                  │
│  │   ╰────────────┴──────────────────────────────────────────────────────╮│
│  ╰──                                                                        │
│  ╭──                                                                        │
│  │ EventID: <event_id>:1                                                    │
│  │ PackageID: 0x<package_id>                                                │
│  │ Transaction Module: registry                                             │
│  │ Sender: 0x<sender_address>                                               │
│  │ EventType: 0x<package_id>::registry::GrantIndexed                        │
│  │ ParsedJSON:                                                              │
│  │   ╭────────────┬──────────────────────────────────────────────────────╮│
│  │   │ grant_id   │ 0x<grant_object_id>                                  │
│  │   ├────────────┼──────────────────────────────────────────────────────╮│
│  │   │ student    │ 0x<student_address>                                  │
│  │   ╰────────────┴──────────────────────────────────────────────────────╮│
│  ╰──                                                                        │
╰─────────────────────────────────────────────────────────────────────────────╯
```

More on composing PTBs: CLI PTB Reference

WHY ATOMIC?

If mint fails, index doesn't run; if index would fail, mint is rolled back. One intent, one commit. This reduces partial failures and keeps state consistent.

CLI PTB ARGUMENT FORMATTING

- Addresses: pass as plain `0x...` (no `@`) or as a variable you assigned earlier. Example: `--move-call ... "0xabc..."` or `--assign student @0xabc...` then `--move-call ... student`.
- Object IDs: pass with `@` (e.g., `@"$ADMIN_CAP_ID"`, `@0x...`) or via a variable you assigned to an object ID.
- Returned values: use `--assign name` to capture a result (e.g., `minted_id`) and pass the variable name without `@`.

Verify outcomes

```
iota client objects --owner $STUDENT_ADDR
iota client object --id $REGISTRY_ID
```

You should see the new grant under the student's ownership and a shared `Registry` with the index populated.

BIG PICTURE

From here, your portal can fetch the grant and the registry mapping to render a student's status instantly. Indexers can subscribe to `GrantIndexed` to drive analytics.

Common pitfalls

CAUTION

- Network: ensure your CLI is connected to Testnet (`iota client envs`; switch with `iota client switch --env testnet`).
- Funding: request test tokens via faucet before publishing/calling (`iota client faucet --address $PUBLISHER_ADDR`).
- Upgrade compatibility: do not change existing public/entry function signatures (e.g., do not make `mint` return a value). Add a new entry like `mint_return_id` instead.

- Upgrading vs publishing new: ensure you reference the correct `PKG_ID`.
- Write-gating: pass the correct `ADMIN_CAP_ID` to `index_grant`.
- Create the `Registry` before trying to index, and capture `REGISTRY_ID`.
- Tables: make sure key type is `address` and value type is `ID`.
- Gas budgets: bump `--gas-budget` if you see out-of-gas failures.

Bring this to the next workshop

- `PKG_ID`
- `REGISTRY_ID`
- `ADMIN_CAP_ID`
- At least one `GRANT_OBJECT_ID`

Further reading

- Shared vs Owned Objects
- Dynamic Fields: Tables & Bags
- Use Programmable Transaction Blocks
- TS SDK: Programmable Transaction Basics
- CLI: PTB Reference

Give Feedback

Tags: Tutorial | Move Smart Contracts | TypeScript

Previous
« Core I: Init and Deploy a Grants package

Next
Core III: Capabilities, Type-State, Abilities & Tests »